

Lab 1 — GPIO: The RGB LED and the BOOT Button

ESP32-S3 FIRMWARE LAB SERIES · PHASE 1 · FOUNDATIONS

Estimated time	2 hrs
Introduces	designated initializers, handles, active-low inputs
Prerequisites	Lab 0 (a working build, flash and monitor loop).
Beyond the board	Nothing — the RGB LED and BOOT button are on the board.
Target	ESP32-S3 (written against an N16R8 module: 16 MB flash, 8 MB octal PSRAM)
Framework	ESP-IDF v5.5, CMake and Ninja, driven by <code>idf.py</code>

NOTE

How to read this lab

Sections run **§1.1 objective**, **§1.2 parts and wiring**, **§1.3 how it works**, then the code, a line-by-line walkthrough, the documentation map, and verification. Callout boxes are tagged **[WARNING]**, **[KEY POINT]**, **[NOTE]**, **[DOCS]** and **[TM4C BRIDGE]**. Figures are given as captioned text and tables so every value in them is searchable and checkable.

The **TM4C bridge** notes map each topic onto the equivalent bare-metal, register-level work on a TI TM4C123 (Cortex-M4F). Skip them if that is not your background; nothing later depends on them.

1.1 Objective & TM4C bridge

Read a button and drive the onboard RGB LED. Along the way, meet the ESP-IDF idioms — config structs, handles, error checking — that every later lab reuses.

TM4C BRIDGE

This is the classic bare-metal GPIO blink with the register writes replaced by driver calls. On the TM4C you enabled the port clock in `RCGCGPIO`, set direction in `DIR`, enabled the digital function in `DEN`, and configured a pull resistor in `PUR`, then read and wrote `GPIO_DATA`. Here a single `gpio_config_t` struct replaces that entire sequence. The registers still exist; the driver writes them for you.

1.2 Parts & wiring

Item	Status	Pin
RGB LED (WS2812 "smart" LED)	onboard	GPIO48
BOOT button	onboard	GPIO0

No wiring at all. Both peripherals are already on the board — and they are the *only* two. From Lab 3 onward you will be wiring external parts, but this lab needs nothing but the USB cable.

NOTE

Why the button is on GPIO0, and why that is safe

GPIO0 is a **strapping pin**: its level at the moment of reset tells the chip whether to boot normally or enter the serial bootloader. That is what the BOOT button is physically for. Once the chip has booted, the pin is an ordinary GPIO and reading it is completely safe. Driving it as an output would be a different matter. This dual-purpose-pin situation is common and always documented in the datasheet's pin description table.

1.3 How it works

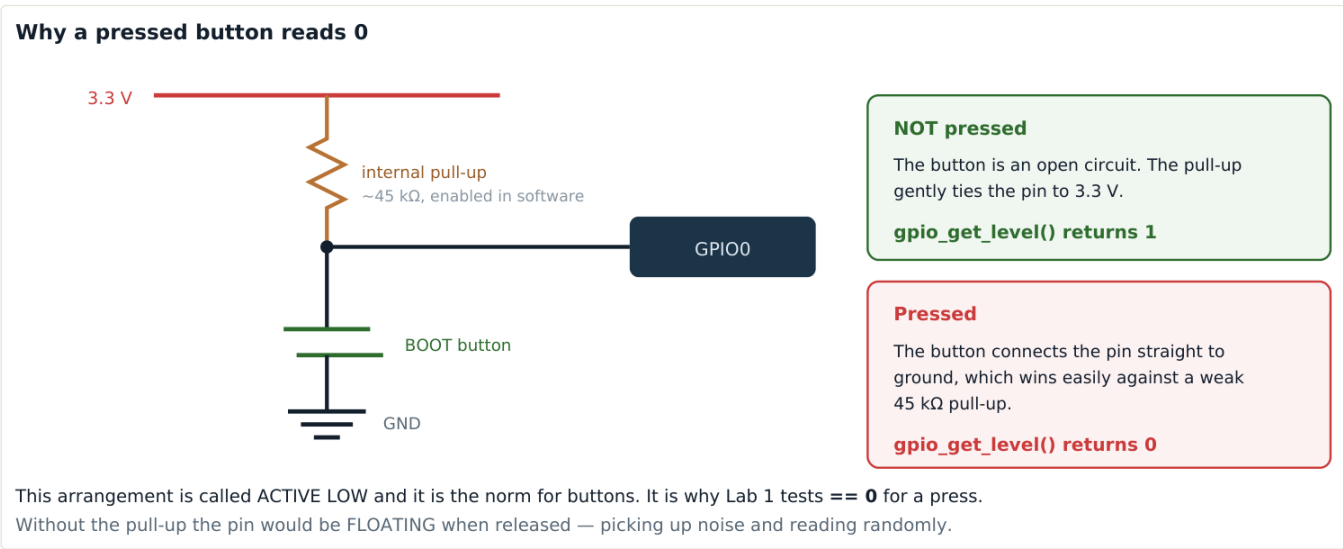


Figure 1.1. The pull-up holds the pin high when nothing else drives it; pressing the button connects the pin to ground, which easily wins against a weak 45 kΩ resistor. That is why a press reads 0.

Active-low button wiring with an internal pull-up resistor.

Circuit: 3.3 V rail -> internal pull-up resistor (about 45 kilohms, enabled in software) -> the node that connects to GPIO0 -> BOOT button -> GND.

Button state	What happens	gpio_get_level() returns
NOT pressed	the button is an open circuit; the pull-up gently ties the pin to 3.3 V	1
Pressed	the button connects the pin straight to ground, which wins easily against a weak 45 kilohm pull-up	0

This arrangement is called **ACTIVE LOW** and it is the norm for buttons. It is why Lab 1 tests `== 0` for a press, which reads backwards exactly once.

Without the pull-up the pin would be **FLOATING** when released, picking up noise and reading randomly.

Inputs need a defined idle level

A pin configured as an input and connected to nothing is **floating** — its voltage drifts with nearby electrical noise and reads randomly as 0 or 1. A button makes this concrete: a simple switch only *connects* two points when pressed, so while it is unpressed the pin is attached to nothing at all.

The fix is a **pull resistor** that gently ties the pin to a known level. With an internal **pull-up** enabled, the pin idles HIGH; pressing the button connects it to ground and drags it LOW. So **pressed reads 0**, which is called **active-low** logic and is the near-universal convention for buttons. It reads backwards the first time and then never again. **TM4C bridge:** this is exactly the **PUR** register and the active-low switches on the LaunchPad.

Why this LED needs a whole peripheral

The onboard RGB LED is a **WS2812**, not a plain LED. A plain LED is just a diode: set the pin high, it lights. A WS2812 contains a tiny controller, and you set its colour by sending a serial data stream of 24 bits — 8 each for green, red and blue — where a bit's *value* is encoded in the *width* of a pulse. A one and a zero differ by a few hundred nanoseconds.

That timing is far too tight to bit-bang reliably from software while an RTOS may preempt you mid-stream. So the ESP32-S3 uses the **RMT** peripheral (Remote Control Transceiver), hardware originally built for infrared remote protocols, which generates precisely-timed pulse sequences without CPU involvement. You hand it the pattern and the hardware clocks it out. That is why Lab 1 pulls in a driver component rather than calling `gpio_set_level`, and it is a good, specific answer if an interviewer asks why you would choose a peripheral over software timing.

Stage, then refresh

Because the colour is a serial transmission rather than a pin level, the driver splits the operation in two: `led_strip_set_pixel` writes the colour into a framebuffer in RAM, and `led_strip_refresh` actually clocks that buffer out over RMT. Nothing appears until you refresh. Forgetting the refresh is the classic Lab 1 bug, and the same stage-then-apply pattern recurs in Lab 6 (PWM) and Lab 10 (NVS).

1.4 Procedure & code

1. Create the project, `idf.py set-target esp32s3`.
2. Add the LED driver component: `idf.py add-dependency "espressif/led_strip"`.
3. Type the listing, then `idf.py build flash monitor`.

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"
#include "led_strip.h"

#define RGB_GPIO 48
#define BOOT_GPIO 0 // the BOOT button. Also a strapping pin -- fine to READ after boot.
```

```

static led_strip_handle_t strip;

static void rgb_init(void) {
    led_strip_config_t sc = {
        .strip_gpio_num = RGB_GPIO,
        .max_leds = 1,
    };
    led_strip_rmt_config_t rc = {
        .resolution_hz = 10 * 1000 * 1000, // 10 MHz RMT tick
    };
    ESP_ERROR_CHECK(led_strip_new_rmt_device(&sc, &rc, &strip));
    led_strip_clear(strip);
}

void app_main(void) {
    rgb_init();
    gpio_config_t btn = { // replaces RCGCGPIO + DIR + DEN + pull setup
        .pin_bit_mask = 1ULL << BOOT_GPIO,
        .mode = GPIO_MODE_INPUT,
        .pull_up_en = GPIO_PULLUP_ENABLE, // button pulls the line LOW when pressed
    };
    gpio_config(&btn);

    bool on = false;
    while (1) {
        if (gpio_get_level(BOOT_GPIO) == 0) { // active-low: pressed == 0
            on = !on;
            // crude debounce: wait for release
            while (gpio_get_level(BOOT_GPIO) == 0) vTaskDelay(pdMS_TO_TICKS(10));
        }
        if (on) led_strip_set_pixel(strip, 0, 16, 0, 0); // index, R, G, B (dim red)
        else led_strip_clear(strip);
        led_strip_refresh(strip); // nothing shows until you refresh
        vTaskDelay(pdMS_TO_TICKS(20));
    }
}

```

1.5 Code walkthrough, line by line

Forty-five lines, and several of the C idioms here recur in every later lab. Nothing in it is throwaway.

Lines 4-5 · `#include "driver/gpio.h" / "led_strip.h"`

`driver/gpio.h` is the GPIO API — the ESP-IDF counterpart of the GPIO section of your TM4C header. `led_strip.h` is the WS2812 driver you pulled in with `add-dependency`, and it is a **managed component** rather than part of core ESP-IDF, which is why its source lands in your project under `managed_components/`.

Lines 7-8 · `#define RGB_GPIO 48 / #define BOOT_GPIO 0`

Naming the pins so no magic numbers float around — the same discipline as your TM4C code. A `#define` is handled by the *preprocessor*: it is textual substitution done before the compiler ever sees the file, which is why there is no semicolon and no type.

Line 10 · `static led_strip_handle_t strip;`

Three separate things worth unpacking, and all three recur constantly.

static at file scope means "this variable is private to this `.c` file" — a visibility control, like `private` in Java. That is *not* the same as the "persists between calls" meaning `static` has *inside* a function. One keyword, two unrelated jobs, and which one applies depends purely on where it appears.

The `_t` suffix is C convention for "this is a type name," like `size_t` or `uint8_t`. It tells you at a glance that `led_strip_handle_t` is a type rather than a variable or function.

A **handle** is a pointer-like opaque token. You never look inside it; you pass it to the library's functions so they know which strip you mean. **TM4C bridge:** on the TM4C you addressed hardware directly by writing registers. Here a driver hands you a handle and you talk to the hardware through it.

Lines 13-16 · `led_strip_config_t sc = { .strip_gpio_num = ..., .max_leds = 1 };`

This is a **designated initializer** (C99): the `.field = value` form fills struct members *by name* rather than by position. Three reasons professional embedded code uses it. It is self-documenting. It stays correct if the library reorders its struct in a future version. And **any field you do not mention is set to zero**, so you only write what you care about — this struct has more members than the two shown.

Read it as: one LED, on pin 48.

Lines 17-19 · `led_strip_rmt_config_t rc = { .resolution_hz = 10 * 1000 * 1000 };`

A second config struct, for the RMT peripheral that will actually generate the waveform. `10 * 1000 * 1000` is 10 MHz written so the magnitude is readable at a glance — the compiler folds the arithmetic at compile time, so it costs nothing. That tick rate is fast enough to hit the WS2812's few-hundred-nanosecond bit timing, which is the whole reason this LED needs RMT hardware instead of a plain pin.

Line 20 · `ESP_ERROR_CHECK(led_strip_new_rmt_device(&sc, &rc, &strip));`

Two things layered together. The inner call creates the device, and the three **address-of** operators (`&`) matter: the function takes pointers so it can *read* your two config structs and *write* the created handle back into `strip`. That third one is an **output parameter**, the standard C way of returning something when the return value is already used for an error code.

`ESP_ERROR_CHECK(...)` wraps a call returning `esp_err_t` and aborts loudly with a message and a backtrace if it is not `ESP_OK`. During bring-up that is exactly what you want — a misconfiguration surfaces immediately instead of silently producing a dead peripheral.

Line 21 · `led_strip_clear(strip);`

Turns the LED off so you start from a known state. Note it takes the handle, not a pin number — from here on, `strip` is the LED as far as your code is concerned.

Line 27 · `.pin_bit_mask = 1ULL << BOOT_GPIO`

The line that needs the most care. The mask is a 64-bit value with **one bit set per pin you are configuring**, so you could configure several pins at once by ORing their masks together. `1ULL << 0` sets bit 0, for GPIO0.

The `ULL` suffix makes the literal an **unsigned long long** (64-bit). This is not decoration: `1 << 40` on a 32-bit `int` is *undefined behaviour*, and this chip has pins above 31. Getting into the habit of matching the literal's width to the field is worth doing everywhere.

Lines 28-29 · `.mode = GPIO_MODE_INPUT / .pull_up_en = GPIO_PULLUP_ENABLE`

`.mode` sets the direction. `.pull_up_en` switches on the internal pull-up so the pin idles HIGH and the button drags it LOW when pressed — the **active-low** arrangement: the resistor gently ties the pin HIGH when nothing else drives it, and pressing the button connects the pin to ground, so *pressed reads 0*. Two fields the struct also has, `.pull_down_en` and `.intr_type`, are left unmentioned and therefore zero; Lab 8 sets that last one.

Line 31 · `gpio_config(&btn);`

Applies the struct. **TM4C bridge:** this single call replaces your entire `RCGC_GPIO` (clock enable) + `DIR` (direction) + `DEN` (digital enable) + `PUR` (pull-up) sequence. The registers still get written; the driver does it.

Line 33 · `bool on = false;`

Tracks the LED state between loop iterations. It is declared *outside* the `while`, which is the whole point — a variable declared inside would be reinitialised every pass and the toggle would never persist.

Line 35 · `if (gpio_get_level(BOOT_GPIO) == 0)`

Reads the pin. The comparison is against **0** because the button is **active-low**: pressed connects the pin to ground, so pressed reads 0 and released reads 1. This reads backwards the first time and never again.

Line 36 · `on = !on;`

The toggle. `!` is logical NOT, so this assigns the opposite of the current value.

WARNING

The typo that compiles cleanly and does nothing

Writing `on != on` instead of `on = !on` is a *comparison*, not an assignment. It evaluates to false, the result is discarded, and the compiler is perfectly happy. The LED simply never changes. One character, no error message.

Line 38 · `while (gpio_get_level(BOOT_GPIO) == 0) vTaskDelay(pdMS_TO_TICKS(10));`

A **crude debounce**. A mechanical switch does not close cleanly — the contacts bounce for several milliseconds, and without this the single press on line 35 would be counted dozens of times. This inner loop waits, polling every 10 ms, until you release the button.

Note it uses `vTaskDelay` rather than a busy-wait, so even while waiting for a human finger it yields the CPU. Lab 8 replaces this whole approach with an interrupt, which is strictly better; this version is here because it is the honest first thing you would write.

Lines 40-41 · `led_strip_set_pixel(strip, 0, 16, 0, 0) / led_strip_clear(strip)`

The arguments are handle, LED index, red, green, blue. Index **0** is the first (and only) LED. The values `16, 0, 0` are dim red — each channel runs 0-255, and 16 is deliberately low because these LEDs are genuinely painful at full brightness.

The key word is **stages**: this call writes the colour into a buffer in RAM and nothing happens on the LED yet.

Line 42 · `led_strip_refresh(strip);`

This is what actually clocks the colour word out over RMT to the LED. **Forgetting this line is the classic Lab 1 bug** — the code runs, no error appears, and the LED never changes. **TM4C bridge**: there, writing `GPIODATA` lit an LED instantly. A smart LED needs the explicit refresh, and the same stage-then-apply pattern comes back in Lab 6 (`ledc_update_duty`) and Lab 10 (`nvs_commit`).

Line 43 · `vTaskDelay(pdMS_TO_TICKS(20));`

Paces the loop at 50 Hz and yields the CPU so this task is not a busy-loop. 20 ms is fast enough that a button press never feels laggy and slow enough that you are not polling pointlessly.

1.6 Docs & references

DOCS

Trace every line to its source

`gpio_config_t`, `gpio_config`, `gpio_get_level` — ESP-IDF API Reference → Peripherals → **GPIO**. Read the whole `gpio_config_t` struct definition; you are using four fields out of six.

`led_strip_*` — this is a **managed component**, not part of core ESP-IDF. Its documentation lives in the ESP Component Registry (components.espressif.com), and its source lands in your project under `managed_components/` after `add-dependency`. Reading a component's header file is often faster than finding its docs.

RMT peripheral — ESP-IDF API Reference → Peripherals → **RMT**, and the RMT chapter of the ESP32-S3 Technical Reference Manual if you want the hardware detail behind §1.3.

Strapping pins — ESP32-S3 Datasheet, pin description section. Confirm GPIO0's boot behaviour for yourself.

1.7 Verify, troubleshoot, your turn

Verify. Each BOOT press flips the LED between dim red and off. Change `(16, 0, 0)` to `(0, 16, 0)` for green or `(0, 0, 16)` for blue. Keep the values low; these LEDs are genuinely painful at 255.

Troubleshoot

Symptom	Cause and fix
<code>led_strip.h</code> not found	The component did not download. Delete <code>managed_components/</code> and <code>dependencies.lock</code> , then <code>idf.py reconfigure</code> and build again.
LED never lights	Missing <code>led_strip_refresh</code> , or the RGB pin is not 48 on your board revision.
One press toggles many times	The debounce loop was removed or the release-wait is too short.
Editor shows red squiggles but the build succeeds	IntelliSense, not the compiler. Trust the terminal, not the editor. This distinction will save you hours across the track.

Your turn

Cycle red, green, blue on successive presses instead of toggling one colour. A `switch` on a counter taken modulo 3 is the clean way. Then try making the LED breathe smoothly — you will find you cannot do it well with delays alone, which is exactly the problem Lab 6 solves with hardware PWM.

The series at a glance

Each lab is a complete, independently buildable ESP-IDF project. This is Lab 1.

#	Lab	Phase
00	Toolchain: hello, ESP32-S3	Foundations
01 ←	GPIO: RGB LED and BOOT button	Foundations
02	FreeRTOS tasks and drift-free timing	Foundations
03	UART and the loopback test	Foundations
04	Hardware timers and the 1 kHz problem	Foundations
05	I ² C and the IMU bring-up	Foundations
06	PWM with the LEDC peripheral	Foundations
07	The sensor pipeline (capstone)	Foundations
08	GPIO interrupts and ISR-safe code	Past the coursework
09	SPI, full-duplex and fast	Past the coursework
10	NVS: data that survives power loss	Past the coursework
11	Unit testing with Unity	Past the coursework
12	Deep sleep and waking up	Past the coursework
13	Wi-Fi station and JSON telemetry	Past the coursework
14	BLE GATT with NimBLE	Past the coursework
15	OTA and the rollback safety net	Past the coursework
16	Priority inversion and mutexes	Past the coursework
17	CAN (TWAI): broadcast by message ID	Past the coursework
18	PID, anti-windup and derivative kick	Past the coursework
19	The ML inference harness	Gap-closers
20	Continuous ADC: DMA-backed acquisition	Gap-closers
21	On-chip debugging: JTAG, OpenOCD, GDB	Gap-closers
22	A real display (SPI TFT + DMA)	Gap-closers
23	Production test mode	Gap-closers

Written against ESP-IDF v5.5 on an ESP32-S3 N16R8 module. Pin assignments are choices, not constraints — the ESP32-S3 GPIO matrix routes almost any peripheral to almost any pin, with the exception of GPIO26–GPIO37, which are committed to the on-board flash and PSRAM and must never be used.